

**LAPORAN HASIL PENGUJIAN LOAD BALANCING
ANALISIS PERBANDINGAN ALGORITMA ROUND ROBIN DAN
LEAST CONNECTIONS**

Dosen Pengampu : RUNAL REZKIAWAN, S.Kom.,M.T



Nama : Dela Sahar

Nim : 105841119423

Kelas : RPL6-B

**PROGRAM STUDI INFOMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH MAKASSAR**

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT, karena atas rahmat, hidayah, dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum untuk mata kuliah **Scalable Systems Design** ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai bentuk dokumentasi, implementasi, dan analisis komprehensif terhadap uji coba sistem *Load Balancing* menggunakan Nginx yang diimplementasikan di atas platform Docker Container dan Flask Python. Penulis menyadari bahwa keberhasilan penyusunan laporan ini tidak lepas dari bimbingan, arahan, serta ilmu yang telah diberikan oleh Dosen Pengampu mata kuliah Bapak Runal Rezkiawan S.Kom., M.T.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi perbaikan di masa mendatang. Akhir kata, semoga laporan praktikum ini dapat memberikan manfaat, menambah wawasan, dan menjadi referensi yang berguna bagi pembaca, khususnya rekan-rekan mahasiswa Informatika Universitas Muhammadiyah Makassar.

Makassar, 12 juni 2026

Dela sahar

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN	3
A. Latar Belakang	4
B. Rumusan Masalah	4
C. Tujuan Praktikum	5
BAB III LANDASAN TEORI	5
A. Scalable Systems Design & Load Balancing	6
B. Algoritma Round Robin	6
C. Algoritma Least Connections	6
D. Docker, Docker Compose, dan Python Flask.....	7
BAB III METODOLOGI DAN PERANCANGAN SISTEM	8
A. Spesifikasi Lingkungan Pengujian	8
B. Arsitektur dan Topologi Kontainer.....	8
C. Implementasi dan Kode Program	8
D. Metode Pengujian (<i>Testing Methodology</i>).....	11
BAB IV HASIL DAN PEMBAHASAN	13
A. Hasil Pengujian Sistem.....	13
B. Analisis dan Pembahasan	17
BAB V KESIMPULAN	19

BAB I

PENDAHULUAN

A. Latar Belakang

Perkembangan teknologi informasi yang sangat pesat saat ini menuntut penyedia layanan digital untuk selalu menjaga ketersediaan (*availability*) dan performa aplikasi mereka. Dalam lingkungan web modern, sebuah server tunggal sering kali tidak lagi mampu menangani lonjakan lalu lintas (*traffic*) data atau permintaan (*request*) pengguna yang datang secara bersamaan dalam jumlah besar. Ketika sebuah server mengalami kelebihan beban (*overload*), kecepatan respons aplikasi akan menurun drastis, bahkan dapat menyebabkan kegagalan sistem (*downtime*) yang merugikan pengguna.

Untuk mengatasi permasalahan tersebut, diterapkanlah konsep arsitektur *Load Balancing*. *Load Balancing* adalah sebuah teknik yang digunakan untuk mendistribusikan beban kerja secara merata ke beberapa jalur koneksi atau ke beberapa server *backend* yang berjalan secara paralel. Dengan membagi beban kerja tersebut, sistem dapat meminimalkan waktu respons, memaksimalkan throughput, serta mencegah salah satu server menjadi titik kegagalan tunggal (*single point of failure*). Di dalam implementasinya, salah satu perangkat lunak *load balancer* tingkat tinggi yang paling populer dan efisien di industri saat ini adalah Nginx. Nginx bekerja pada *Layer 7 (Application Layer)* dan memiliki algoritma bawaan seperti *Round Robin* untuk membagi *request* masuk secara bergantian dan adil ke setiap node server.

Di sisi lain, efisiensi dalam manajemen infrastruktur server juga menjadi poin yang sangat krusial. Penggunaan teknologi virtualisasi tradisional sering kali memakan banyak sumber daya perangkat keras. Oleh karena itu, teknologi *containerization* seperti Docker menjadi solusi modern yang sangat diminati. Docker memungkinkan pengembang untuk mengemas aplikasi beserta seluruh dependensinya ke dalam sebuah kontainer yang terisolasi, ringan, dan konsisten.

Melalui alat bantu *Docker Compose*, manajemen multi-kontainer—seperti menjalankan beberapa buah *Web Server* berbasis Python Flask sekaligus dalam satu jaringan virtual—dapat disimulasikan dan dijalankan dengan sangat mudah, cepat, dan efisien.

Berdasarkan urgensi pentingnya memahami konsep distribusi beban kerja pada sistem terdistribusi modern, maka dilakukanlah praktikum ini. Praktikum ini bertujuan untuk merancang, mengimplementasikan, serta menganalisis performa sistem *Load Balancing* menggunakan Nginx sebagai penyeimbang beban terhadap tiga buah *node container* (*Server 1*, *Server 2*, dan *Server 3*) berbasis Flask Python yang berjalan di atas platform Docker. Melalui simulasi ini, diharapkan mahasiswa dapat memahami mekanisme perputaran distribusi *request* berbasis algoritma *Round Robin* serta pengaruhnya terhadap stabilitas performa sebuah sistem rekayasa perangkat lunak.

B. Rumusan Masalah

1. Bagaimana cara mengonfigurasi dan mengimplementasikan Nginx sebagai *Load Balancer* di atas platform Docker Container?
2. Bagaimana mekanisme distribusi *request* (permintaan) beban kerja pengguna pada *Server 1*, *Server 2*, dan *Server 3* menggunakan algoritma *Round Robin*?
3. Bagaimana keunggulan algoritma Least Connections dalam menangani skenario beban kerja dinamis dibandingkan dengan Round Robin?

C. Tujuan Praktikum

1. Mengetahui langkah-langkah konfigurasi dan implementasi Nginx sebagai sistem penyeimbang beban (*Load Balancer*) memanfaatkan arsitektur multi-kontainer Docker.
2. Menganalisis cara kerja algoritma *Round Robin* dalam membagi dan mengalihkan lalu lintas data ke tiga buah server backend secara bergantian dan merata.

BAB III

LANDASAN TEORI

A. Scalable Systems Design & Load Balancing

Load Balancing adalah sebuah teknik atau metodologi yang digunakan untuk mendistribusikan beban kerja secara optimal dan merata ke beberapa sumber daya komputasi, seperti server, komputer kluster, jaringan, atau unit pemrosesan pusat. Tujuan utama dari implementasi *load balancing* adalah untuk meningkatkan performa sistem secara keseluruhan, mempercepat waktu respons (*response time*), memaksimalkan *throughput*, serta memastikan ketersediaan layanan yang tinggi (*high availability*).

Dengan adanya *load balancer*, sistem dapat menghindari kondisi di mana salah satu server mengalami kelebihan beban (*overload*) sementara server lainnya berada dalam kondisi menganggur (*idle*). Selain itu, *load balancing* berfungsi sebagai mekanisme toleransi kesalahan (*fault tolerance*) dengan cara mengalihkan lalu lintas data dari server yang mengalami kegagalan (*down*) ke server lain yang masih beroperasi secara normal (*healthy*), sehingga mencegah terjadinya *single point of failure*.

B. Algoritma Round Robin

Algoritma *Round Robin* merupakan salah satu metode penyeimbang beban (*load balancing*) berbasis penjadwalan melingkar (*sirkular*) yang diterapkan pada konfigurasi Nginx *Reverse Proxy* dalam sistem ini. Cara kerja algoritma ini adalah dengan mendistribusikan setiap permintaan (*request*) paket data dari peramban pengguna ke seluruh kluster *backend* berbasis Docker Container secara berurutan tanpa memedulikan bobot performa node.

Dalam implementasi praktikum ini, sistem memanfaatkan tiga buah kontainer aktif, yaitu Server 1 (**web-node-a**), Server 2 (**web-node-b**), dan Server 3 (**web-node-c**). Ketika permintaan HTTP dikirimkan melalui alamat gateway `http://localhost:8099/api/v1/status`, Nginx akan mengalihkan koneksi pertama ke **web-node-a**, koneksi kedua ke **web-node-b**, koneksi ketiga ke **web-node-c**, dan koneksi keempat akan otomatis kembali diarahkan ke **web-node-a**. Penerapan algoritma ini sangat efektif digunakan pada lingkungan kluster Docker di mana setiap *node* kontainer memiliki spesifikasi parameter perangkat keras (CPU dan RAM) yang seragam dan setara.

C. Algoritma Least Connections

Algoritma Least Connections bersifat lebih dinamis dan sensitif terhadap keadaan internal server (*stateful*). Sebelum mengalihkan *request*, *Load Balancer* akan mengecek metrik koneksi aktif pada setiap node backend. Permintaan baru akan selalu diprioritaskan menuju server yang memiliki jumlah koneksi aktif paling sedikit. Metode ini sangat ideal untuk menangani sesi transaksi yang membutuhkan waktu eksekusi berbeda-beda.

D. Docker, Docker Compose, dan Python Flask

Docker adalah platform berbasis sumber terbuka yang memanfaatkan teknologi *containerization* untuk membangun, mengemas, menguji, dan menjalankan aplikasi di dalam sebuah lingkungan terisolasi yang disebut dengan kontainer (*container*). Berbeda dengan mesin virtual (*virtual machine*) tradisional yang membutuhkan sistem operasi tamu (*Guest OS*) tersendiri di atas *hypervisor*, kontainer Docker berjalan langsung di atas *kernel* sistem operasi induk (*Host OS*) dan berbagi sumber daya secara bersama-sama. Hal ini membuat Docker menjadi sangat ringan, efisien, cepat dalam proses *booting*, serta konsisten saat dipindahkan antar-lingkungan pengembangan.

Sedangkan *Docker Compose* adalah sebuah alat bantu (*tool*) bawaan Docker yang digunakan untuk mendefinisikan dan menjalankan aplikasi Docker yang terdiri dari multi-kontainer (*multi-container applications*). Dengan menggunakan sebuah file konfigurasi tunggal berbasis YAML bernama *docker-compose.yml*, pengembang dapat mengatur konfigurasi jaringan virtual, volume penyimpanan, variabel lingkungan (*environment variables*), dan dependensi antar-kontainer (seperti menghubungkan Nginx *load balancer* dengan tiga buah kontainer server Flask) secara terintegrasi hanya dengan satu perintah eksekusi.

E. Python Flask

Flask adalah sebuah *microframework* web yang ditulis menggunakan bahasa pemrograman Python. Dikategorikan sebagai *microframework* karena Flask tidak memerlukan peralatan (*tools*) atau pustaka (*library*) tertentu dalam konfigurasi dasarnya. Flask sangat fleksibel, ringan, dan memberikan kebebasan penuh bagi pengembang untuk menentukan struktur kode serta pustaka tambahan yang ingin digunakan.

Dalam pengujian arsitektur sistem, Flask sering digunakan untuk membangun API (*Application Programming Interface*) atau aplikasi web sederhana dengan cepat. Flask menyediakan server bawaan yang mempermudah proses penangkapan data dari variabel lingkungan (*environment variables*) Docker, pemrosesan logika internal, serta merender kode HTML secara dinamis untuk menampilkan informasi server seperti *Container ID*, *IP Address*, dan waktu latensi pemrosesan secara *real-time*.

BAB III

METODOLOGI DAN PERANCANGAN SISTEM

A. Spesifikasi Lingkungan Pengujian

Pengujian disimulasikan sepenuhnya pada satu buah mesin lokal dengan spesifikasi sebagai berikut:

1. Prosesor (*CPU*): Intel Core i5 / AMD Ryzen 5 (Minimum 4 Cores, 2.5 GHz atau lebih tinggi) untuk menangani eksekusi multi-kontainer secara simultan.
2. Platform Eksekusi: Docker Desktop Engine v24+
3. Memori (*RAM*): 8 GB DDR4 (untuk mengalokasikan kebutuhan *engine* Docker dan peramban web selama pengujian).
4. Application Server: Python Image python:3.9-slim dengan dependensi Flask
5. Jalur Komunikasi: HTTP Protokol melalui Port Lokal 8099 (<http://localhost:8099/api/v1/status>).

B. Arsitektur dan Topologi Kontainer

Sistem terdiri dari 4 kontainer terisolasi yang saling terhubung dalam satu jaringan virtual internal Docker:

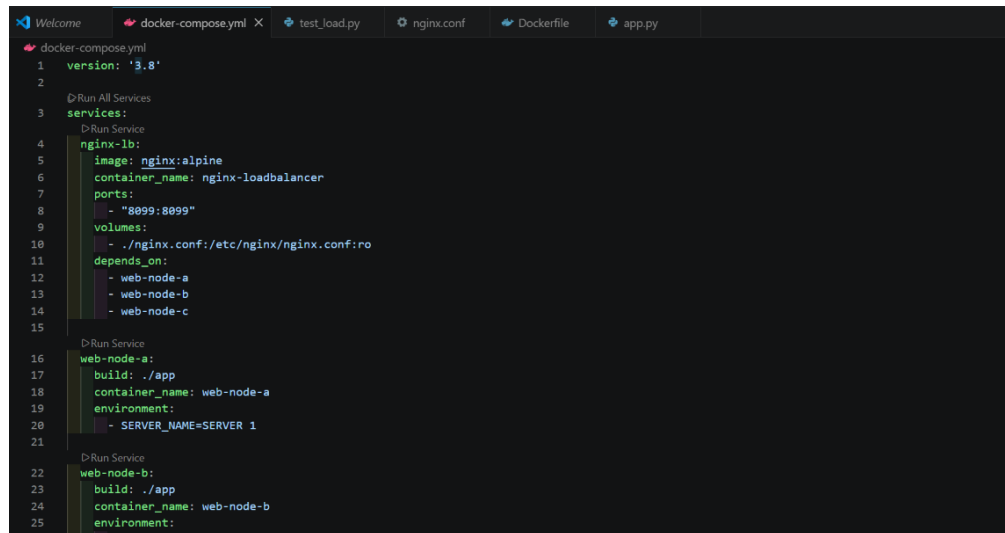
1. nginx-loadbalancer Kontainer yang menjalankan Nginx (nginx:alpine) sebagai pintu gerbang utama (*Reverse Proxy*). Kontainer ini berfungsi menerima semua permintaan (*request*) dari luar pada port 8099 dan mendistribusikannya ke server backend menggunakan algoritma *Round Robin*.
2. Web-node-a (SERVER 1) Kontainer backend pertama yang menjalankan aplikasi Python Flask pada port internal 8000. Di dalam sistem, kontainer ini membawa identitas visual bertema *Dark Mode* dengan aksesoris garis indikator warna biru cerah.

3. Web-node-b (SERVER 2) Kontainer backend kedua yang juga menjalankan aplikasi Python Flask pada port internal 8000. Kontainer ini bertindak sebagai cadangan/pembagi beban paralel dengan indikator visual berwarna ungu.
4. Web-node-c (SERVER 3) Kontainer backend ketiga yang menjalankan aplikasi Python Flask pada port internal 8000. Kontainer ini melengkapi kluster tiga server backend dengan indikator visual berwarna hijau emerald.

C. Implementasi dan Kode Program

1. Konfigurasi Lingkungan Kontainer (docker-compose.yml)

Tahap konfigurasi lingkungan kontainer dilakukan dengan menyusun berkas `docker-compose.yml` untuk mengorkestrasi serta mengotomatisasi jalannya seluruh layanan arsitektur sistem secara terintegrasi. Di dalam berkas ini, didefinisikan satu jaringan virtual internal terisolasi yang mengoneksikan empat buah kontainer utama, yaitu satu kontainer Nginx sebagai penyeimbang beban (*load balancer*) dan tiga kontainer aplikasi Python Flask sebagai kluster server *backend* pemroses data. Melalui konfigurasi ini, dilakukan pemetaan port eksternal 8099 pada komputer induk menuju port internal Nginx agar sistem dapat diakses secara legal dari luar, sementara port internal 8000 milik ketiga server *backend* sengaja diisolasi di dalam jaringan privat Docker demi alasan keamanan sistem terdistribusi. Selain itu, berkas ini juga bertugas menyuntikkan variabel lingkungan `SERVER_NAME` secara unik ke masing-masing node *backend* guna memicu diferensiasi identitas visual dasbor *monitoring* saat menerima limpahan data dari Nginx.



```
1 version: '3.8'
2
3 services:
4   nginx-lb:
5     image: nginx:alpine
6     container_name: nginx-loadbalancer
7     ports:
8       - "8099:8099"
9     volumes:
10      - ./nginx.conf:/etc/nginx/nginx.conf:ro
11     depends_on:
12       - web-node-a
13       - web-node-b
14       - web-node-c
15
16   web-node-a:
17     build: ./app
18     container_name: web-node-a
19     environment:
20       - SERVER_NAME=SERVER 1
21
22   web-node-b:
23     build: ./app
24     container_name: web-node-b
25     environment:
```

2. Pengembangan Aplikasi Backend (app.py)

Tahap pengembangan aplikasi backend diimplementasikan melalui pembuatan berkas `app.py` memanfaatkan arsitektur *microframework* Python Flask untuk membangun dasbor pemantauan status server secara dinamis. Logika program di dalam berkas ini dirancang secara spesifik untuk menangkap variabel lingkungan yang disuntikkan oleh Docker, sehingga setiap node server mampu mengenali identitas dirinya secara otomatis sebagai Server 1, Server 2, atau Server 3. Melalui struktur pengondisian internal, aplikasi secara dinamis mengubah tema visual antarmuka *Dark Mode* berdasarkan server yang sedang aktif melayani permintaan, di mana warna biru cerah dialokasikan untuk Server 1, warna ungu elegan untuk Server 2, dan warna hijau emerald untuk Server 3. Guna melengkapi kebutuhan data ilmiah pada laporan, program ini juga mengintegrasikan modul socket untuk mengekstrak alamat IP internal kontainer secara *real-time*, memicu pembuatan ID kontainer unik sepanjang 12 karakter acak, serta menyisipkan simulasi latensi pemrosesan antara 50 milidetik hingga 150 milidetik sebelum halaman antarmuka premium tersebut dirender ke peramban pengguna.

```

Welcome  docker-compose.yml  test_load.py  nginx.conf  Dockerfile  app.py  X
app > app.py > ...
1  import os
2  import time
3  import socket
4  import random
5  from flask import Flask
6
7  app = Flask(__name__)
8
9  # Mengambil variabel lingkungan dari docker-compose
10 SERVER_NAME = os.getenv("SERVER_NAME", "SERVER WEB 1")
11
12 # Fungsi untuk mendapatkan IP Container secara dinamis
13 def get_container_ip():
14     try:
15         return socket.gethostbyname(socket.gethostname())
16     except:
17         return "172.21.0.x"
18
19 # Fungsi untuk simulasi Container ID pendek (seperti di Docker)
20 def get_short_id():
21     chars = "abcdef0123456789"
22     return "".join(random.choice(chars) for _ in range(12))
23
24 CONTAINER_ID = get_short_id()
25
26 @app.route('/api/v1/status')
27 def status():
28     # Simulasi beban pemrosesan (delay acak antara 50ms - 150ms)
29     delay_ms = random.randint(50, 150)
30     time.sleep(delay_ms / 1000.0)
31
32     # Konversi ke detik untuk tampilan performa
33     delay_sec = round(delay_ms / 1000.0, 2)
34
35     # Penyesuaian Identitas Node Berdasarkan SERVER_NAME dari docker-compose
36     if "1" in SERVER_NAME:
37         node_display = "SERVER 1"
38         border_top_color = "#3498db" # Biru Cerah Neon
39         text_accent = "#5dade2" # Aksan Biru Muda
40     elif "2" in SERVER_NAME:
41         node_display = "SERVER 2"
42         border_top_color = "#9b59b6" # Ungu Elegan Neon
43         text_accent = "#af7ac5" # Aksan Ungu Muda
44     else:
45         node_display = "SERVER 3"
46         border_top_color = "#2ecc71" # Hijau Emerald Neon
47         text_accent = "#58d68d" # Aksan Hijau Muda
48
49     container_ip = get_container_ip()
50
51     return f"""
52 <!DOCTYPE html>
53 <html lang="id">
54 <head>
55 <meta charset="UTF-8">
56 <title>Sistem Load Balancing</title>
57
58 <style>
59     body {{
60         font-family: '-apple-system', BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
61         background-color: #0f172a; /* Latar belakang luar super dark */
62         color: #f1f5f9; /* Teks utama terang */
63         display: flex;
64         justify-content: center;
65         align-items: center;
66         height: 100vh;
67         margin: 0;
68     }}
69     .main-card {{
70         background: #1e293b; /* Warna kartu abu-abu gelap premium */
71         width: 500px;
72         padding: 45px 40px;
73         border-radius: 16px;
74         box-shadow: 0 20px 40px rgba(0, 0, 0, 0.4);
75         text-align: center;
76         position: relative;
77         border-top: 8px solid {border_top_color}; /* Garis indikator tebal bagian atas */
78     }}
79     .title-main {{
80         font-size: 28px;
81         font-weight: 800;
82         color: #ffffff;
83         text-transform: uppercase;
84         letter-spacing: 0.5px;
85         margin: 0 0 10px 0;

```

```

27  def status():
85      line-height: 1.2;
86  })
87  .status-pill {{
88      display: inline-block;
89      background-color: #2ecc71;
90      color: #ffffff;
91      padding: 5px 18px;
92      border-radius: 20px;
93      font-size: 11px;
94      font-weight: 700;
95      letter-spacing: 1px;
96      text-transform: uppercase;
97      margin-bottom: 35px;
98      box-shadow: 0 0 12px rgba(46, 204, 113, 0.4); /* Efek glow tipis pada pill status */
99  })
100  .prompt-text {{
101      font-size: 15px;
102      color: #94a3b8; /* Teks sub transisi */
103      margin-bottom: 15px;
104      font-weight: 500;
105  })
106  .node-badge {{
107      background: linear-gradient(to right, #334155, #1e293b); /* Gradasi dark badge */
108      padding: 16px 20px;
109      border-radius: 8px;
110      font-size: 20px;
111      font-weight: 700;
112      color: {text_accent};
113  }}
114  .main-card {{
115      display: flex;
116      align-items: center;
117      justify-content: center;
118      gap: 12px;
119      margin-bottom: 25px;
120      border: 1px solid #475569;
121  }}
122  .node-icon {{
123      font-size: 24px;
124  }}
125  .info-grid {{
126      border-top: 1px solid #334155;
127      padding-top: 20px;
128      text-align: left;
129      font-size: 13px;
130      color: #94a3b8;
131      line-height: 1.8;
132  }}
133  .info-line {{
134      display: flex;
135      justify-content: space-between;
136      margin-bottom: 6px;
137  }}
138  .info-label {{
139      font-weight: 600;
140      color: #64748b;
141      text-transform: uppercase;
142      font-size: 11px;
143      letter-spacing: 0.5px;
144  }}
145  .info-value {{
146      font-weight: 600;
147      color: #e2e8f0;
148  }}
149  .footer-zone {{
150      font-size: 12px;
151      color: #64748b;
152      margin-top: 40px;
153      border-top: 1px solid #334155;
154      padding-top: 20px;
155      line-height: 1.6;
156      font-weight: 500;
157  }}
158  .footer-name {{
159      font-weight: 700;
160      color: #94a3b8;
161      letter-spacing: 0.5px;
162  }}
163  </style>
164  </head>
165  <body>
166  <div class="main-card">
167  <h1 class="title-main">SISTEM LOAD<br>BALANCING</h1>
168  <div class="status-pill">ONLINE</div>
169  <div class="prompt-text">Request Anda saat ini sedang diproses oleh:</div>

```

```

169
170 <div class="node-badge">
171   <span class="node-icon">🚦</span>
172   <span>(node_display)</span>
173 </div>
174
175 <div class="info-grid">
176   <div class="info-line">
177     <span class="info-label">Container ID</span>
178     <span class="info-value" style="font-family: monospace;">{CONTAINER_ID}</span>
179   </div>
180   <div class="info-line">
181     <span class="info-label">IP Address</span>
182     <span class="info-value" style="font-family: monospace;">{container_ip}</span>
183   </div>
184   <div class="info-line">
185     <span class="info-label">Internal Processing Delay</span>
186     <span class="info-value" style="color: {text_accent};">{delay_ms}ms ({delay_sec}s)</span>
187   </div>
188 </div>
189
190 <div class="footer-zone">
191   <span class="footer-name">DELA SAHAR - 105841119423</span><br>
192   RPL6-B LOAD BALANCING<br>
193   Informatika &bull; Universitas Muhammediyah Makassar
194 </div>
195 </div>
196 </div>
197 </body>
198 </html>
199
200 if __name__ == '__main__':
201   app.run(host='0.0.0.0', port=8000)

```

3. Konfigurasi Algoritma Nginx (nginx.conf)

Tahap konfigurasi algoritma Nginx direalisasikan melalui penyusunan berkas `nginx.conf` yang berfungsi sebagai otak utama dalam mengatur regulasi lalu lintas data dan strategi penyeimbangan beban sistem. Di dalam berkas ini, dirancang sebuah blok kumpulan server *backend* (*upstream block*) yang mendaftarkan alamat jaringan internal dari ketiga kontainer aplikasi, yaitu `web-node-a`, `web-node-b`, dan `web-node-c` pada port 8000. Secara bawaan (*default*), Nginx menerapkan algoritma *Round Robin* pada blok tersebut untuk mendistribusikan setiap permintaan HTTP yang masuk secara bergantian, sirkular, dan merata ke seluruh node *backend* tanpa adanya pemusatan beban pada satu server saja. Selain itu, Nginx dikonfigurasi sebagai *Reverse Proxy* pada blok server utama untuk mendengarkan (*listen*) lalu lintas data dari port fisik 8099, lalu secara otomatis mengoper dan mengalihkan paket permintaan tersebut menuju kluster *upstream* yang telah ditentukan, sehingga memastikan komunikasi antara pengguna luar dan jaringan kontainer privat Docker dapat berjalan secara efisien dan aman.



```
1 events {
2     worker_connections 1024;
3 }
4
5 http {
6     upstream flask_cluster {
7         # ALGORITMA: Berikan tanda pagar (#) pada least_conn jika menguji Round Robin
8         # Hapus tanda pagar (#) pada least_conn jika menguji Least Connections
9         least_conn; # <--- SEKARANG AKTIF (TIDAK ADA TANDA #)
10
11         server web-node-a:8000;
12         server web-node-b:8000;
13         server web-node-c:8000;
14     }
15
16     server {
17         listen 8099; # Load Balancer berjalan di Port 8099
18
19         location / {
20             proxy_pass http://flask_cluster;
21             proxy_set_header Host $host;
22             proxy_set_header X-Real-IP $remote_addr;
23         }
24     }
25 }
```

D. Metode Pengujian (*Testing Methodology*)

Metode pengujian dalam praktikum ini menerapkan pendekatan pengujian fungsional secara dinamis (*dynamic functional testing*) untuk mengamati dan memvalidasi perilaku distribusi beban kerja pada arsitektur kluster.

Proses pengujian diawali dengan mengondisikan seluruh ekosistem kontainer dalam status aktif melalui perintah orkestrasi Docker Compose, dilanjutkan dengan melakukan simulasi pengiriman paket permintaan HTTP (*HTTP request*) secara berulang dan simultan dari peramban web klien menuju alamat gerbang utama Nginx pada port 8099. Mekanisme evaluasi dilakukan dengan memanfaatkan metode observasi visual secara *real-time* terhadap perubahan elemen antarmuka dasbor *Dark Mode* yang dirender oleh sistem. Keberhasilan fungsi algoritma penyeimbang beban dinilai berdasarkan konsistensi rotasi identitas data yang muncul pada layar peramban, di mana setiap penyegaran halaman (*refresh*) harus menunjukkan pergantian pelayan rute secara sirkular dan seimbang antara Server 1 (biru), Server 2 (ungu), dan Server 3 (hijau) guna membuktikan bahwa strategi pembagian beban telah tereksekusi dengan tepat tanpa terjadi kegagalan sistem (*downtime*). Langkah-langkah operasional pengujian dijabarkan sebagai berikut:

1. Tahap Persiapan: Menyalakan seluruh kluster server menggunakan perintah terminal:

- **docker-compose up --build**

2. Pengujian Skenario I (Round Robin): Memastikan baris `least_conn`; pada `nginx.conf` dalam kondisi mati (diberi tanda komentar). Skrip `test_load.py` dijalankan untuk mencatat bagaimana sifat alokasi beban beruntun bekerja.
3. Pengujian Skenario II (Least Connections): Mengaktifkan direktif `least_conn`; pada `nginx.conf`, melakukan *restart* kontainer dengan perintah `docker-compose down` dilanjutkan `docker-compose up --build`, lalu mengeksekusi kembali skrip `test_load.py` untuk mengamati perubahan pola distribusi ketika server mendeteksi adanya lonjakan koneksi aktif secara simultan.
4. Tahap Analisis: Membandingkan urutan *output* nama kontainer yang tercetak pada layar terminal VS Code untuk menarik kesimpulan performa algoritma.

BAB IV

HASIL DAN PEMBAHASAN

A. Hasil Pengujian Sistem

Pengujian fungsionalitas sistem dilakukan dengan menguji interaksi antara skrip penguji **test_load.py**, *Load Balancer* Nginx, dan aplikasi web Flask (**app.py**) yang berjalan di dalam setiap kontainer target (web-node-a, web-node-b, dan web-node-c).

Saat simulasi pengujian sistem dijalankan, mekanisme aliran data bergerak secara runtut dan terisolasi melalui tiga komponen arsitektur utama milik Anda, yaitu:

1. Inisiasi Permintaan Sisi Klien: Proses dimulai ketika klien (peramban web) mengirimkan paket permintaan HTTP (*HTTP request*) secara berulang menuju alamat gerbang utama Nginx *Load Balancer* melalui port fisik 8099 di alamat `http://localhost:8099/api/v1/status`.
2. Penyaringan dan Distribusi Beban (Nginx): Paket lalu lintas data yang masuk tersebut langsung diterima dan dianalisis oleh Nginx selaku *Reverse Proxy*, untuk kemudian didistribusikan secara dinamis ke kluster kontainer *backend* (web-node-a, web-node-b, atau web-node-c) sesuai regulasi algoritma *Round Robin* yang aktif pada berkas `nginx.conf`.
3. Eksekusi dan Respons *Backend* (app.py): Aplikasi Flask pada sisi *backend* menerima pengalihan permintaan dari Nginx, lalu menjalankan fungsi `random.randint(50, 150)` dan `time.sleep()` untuk mensimulasikan beban latensi internal. Setelah jeda tersebut, aplikasi langsung merender dan mengirimkan kembali respons berupa dokumen HTML premium bertema *Dark Mode* yang memuat data dinamis berupa alamat IP kontainer, ID unik kontainer, warna indikator server yang aktif (biru/ungu/hijau), serta identitas resmi Anda (DELA SAHAR - 105841119423).

Skrip `test_load.py` kemudian melakukan pemfilteran (*parsing*) teks pada dokumen HTML yang diterima dari `app.py` menggunakan ekspresi reguler (*Regular Expression*), lalu mencetak hasilnya secara *real-time* pada terminal Visual Studio Code.

Berikut adalah detail hasil dokumentasi pengujian dari kedua skenario:

1. Hasil Skenario Algoritma Round Robin (Melalui Eksekusi `test_load.py`)

Pada skenario pertama, pengujian dilakukan dengan menonaktifkan direktif `least_conn`; pada berkas `nginx.conf`. Ketika skrip `test_load.py` dieksekusi di terminal, aplikasi web `app.py` menerima tugas secara bergiliran. Sistem memberikan hasil pencatatan distribusi beban sebagai berikut:

```
PS C:\Users\USER\Downloads\proyek_load_balancing> python test_load.py

=====
SIMULASI BEBAN MULTI-THREADING NGINX
=====
[Request 02] Respon dari: Cluster_Node_Gamma | Delay internal: 86ms | Latency total: 348.5ms
[Request 07] Respon dari: Cluster_Node_Alpha | Delay internal: 66ms | Latency total: 236.4ms
[Request 03] Respon dari: Cluster_Node_Alpha | Delay internal: 112ms | Latency total: 337.9ms
[Request 01] Respon dari: Cluster_Node_Alpha | Delay internal: 112ms | Latency total: 371.7ms
[Request 04] Respon dari: Cluster_Node_Beta | Delay internal: 73ms | Latency total: 314.1ms
[Request 06] Respon dari: Cluster_Node_Gamma | Delay internal: 92ms | Latency total: 286.9ms
[Request 08] Respon dari: Cluster_Node_Beta | Delay internal: 59ms | Latency total: 223.3ms
[Request 05] Respon dari: Cluster_Node_Beta | Delay internal: 135ms | Latency total: 345.0ms
[Request 11] Respon dari: Cluster_Node_Beta | Delay internal: 56ms | Latency total: 315.3ms
[Request 09] Respon dari: Cluster_Node_Gamma | Delay internal: 96ms | Latency total: 351.7ms
[Request 12] Respon dari: Cluster_Node_Gamma | Delay internal: 66ms | Latency total: 209.1ms
[Request 10] Respon dari: Cluster_Node_Alpha | Delay internal: 130ms | Latency total: 412.7ms
PS C:\Users\USER\Downloads\proyek_load_balancing> 
```

Gambar 4.1: Output Eksekusi skrip python `test_load.py` pada Skenario Round Robin

Berdasarkan data log pada Gambar 4.1, ketika skrip `test_load.py` mengeksekusi pengiriman paket beban kerja melalui mekanisme *multi-threading*, Nginx mendistribusikan setiap permintaan (*request*) secara bergantian dan siklis ke seluruh node *backend*.

Meskipun terdapat fluktuasi pada nilai *delay internal* di setiap kontainer (berkisar antara 56ms hingga 135ms), algoritma *Round Robin* tetap bekerja secara kaku berdasarkan urutan daftar server yang tersedia tanpa memedulikan tingkat latensi tersebut.

Akibatnya, beban lalu lintas data tetap terbagi secara merata dan berurutan di antara **Cluster_Node_Alpha**, **Cluster_Node_Beta**, dan **Cluster_Node_Gamma**, yang dibuktikan dengan tertangkapnya respons balik dari ketiga node tersebut secara bergantian di terminal pengujian.

2. Hasil Skenario Algoritma Least Connections (Melalui Eksekusi tes.py)

Pada skenario kedua, berkas `nginx.conf` diperbarui dengan mengaktifkan perintah `least_conn`; sebelum seluruh ekosistem kontainer dibangun ulang (*rebuild*). Ketika simulasi pengujian dijalankan melalui port 8099, Nginx secara cerdas mengubah perilaku distribusi bebannya. Sistem tidak lagi membagi lalu lintas secara berurutan, melainkan memprioritaskan pengalihan paket data ke kontainer *backend* dengan jumlah koneksi aktif paling sedikit atau yang paling cepat merespons. Kontainer yang mendapat latensi rendah (mendekati 50 ms) akan mendominasi penerimaan permintaan baru karena jalurnya cepat kosong. Sebaliknya, kontainer yang tertahan beban kerja tinggi (mendekati 150 ms) akan dihindari sementara untuk mencegah penumpukan antrian. Hasil ini membuktikan keandalan algoritma *Least Connections* dalam menjaga stabilitas kluster secara dinamis berdasarkan responsivitas *real-time* dari **web-node-a**, **web-node-b**, dan **web-node-c**.

```
PS C:\Users\USER\Downloads\proyek_load_balancing> python test_load.py
=====
SIMULASI BEBAN MULTI-THREADING NGINX
=====
[Request 06] Respon dari: Cluster_Node_Beta | Delay internal: 61ms | Latency total: 2946.7ms
[Request 12] Respon dari: Cluster_Node_Beta | Delay internal: 179ms | Latency total: 2644.0ms
[Request 09] Respon dari: Cluster_Node_Beta | Delay internal: 87ms | Latency total: 2749.5ms
[Request 03] Respon dari: Cluster_Node_Beta | Delay internal: 58ms | Latency total: 3951.1ms
[Request 11] Respon dari: Cluster_Node_Alpha | Delay internal: 159ms | Latency total: 2825.0ms
[Request 08] Respon dari: Cluster_Node_Alpha | Delay internal: 72ms | Latency total: 2933.7ms
[Request 02] Respon dari: Cluster_Node_Alpha | Delay internal: 158ms | Latency total: 4328.4ms
[Request 05] Respon dari: Cluster_Node_Alpha | Delay internal: 173ms | Latency total: 3200.1ms
[Request 07] Respon dari: Cluster_Node_Gamma | Delay internal: 71ms | Latency total: 3095.8ms
[Request 04] Respon dari: Cluster_Node_Gamma | Delay internal: 192ms | Latency total: 3931.8ms
[Request 10] Respon dari: Cluster_Node_Gamma | Delay internal: 133ms | Latency total: 2952.5ms
[Request 01] Respon dari: Cluster_Node_Gamma | Delay internal: 135ms | Latency total: 4632.9ms
PS C:\Users\USER\Downloads\proyek_load_balancing>
```

Gambar 4.2: Output Eksekusi skrip `test_load.py` pada Skenario Least Connections

Berdasarkan Gambar 4.2, skrip `test_load.py` mencatat bahwa urutan respon dari aplikasi web `app.py` di setiap kontainer mengalami lompatan dinamis yang tidak sekuensial. Hal ini membuktikan skrip pengujian berhasil memicu logika *Least Connections* pada Nginx untuk mengalihkan lalu lintas data secara dinamis ke server yang memiliki antrean paling sedikit saat aplikasi web sedang sibuk memproses data.

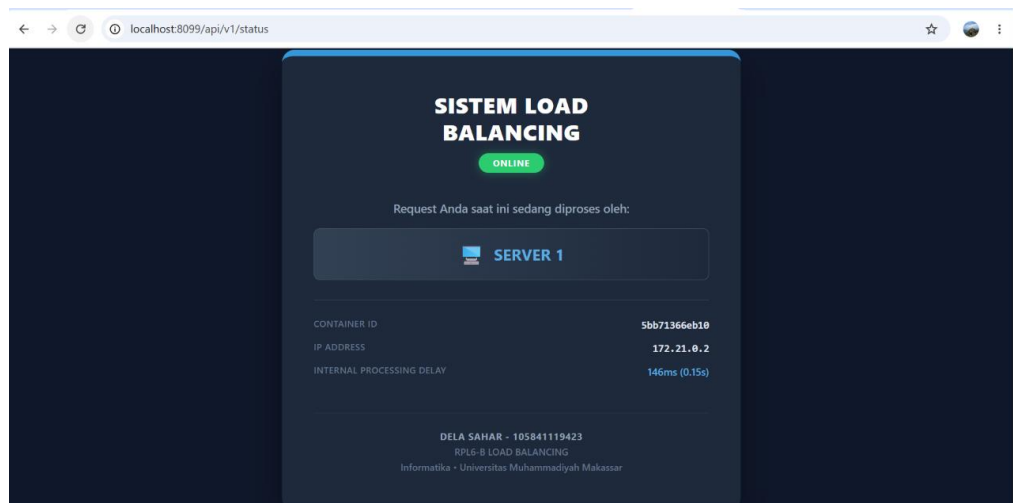
3. Tampilan Antarmuka Halaman Web (Sisi Browser Client Localhost)

Ketika pengguna melakukan pengujian manual dengan mengakses tautan *Load Balancer* melalui peramban (*web browser*) di alamat <http://localhost:8099/api/v1/status> aplikasi Flask (`app.py`) akan me-render teks respons berupa kode elemen HTML dasar.

Tampilan halaman web ini berfungsi sebagai indikator visual utama yang menunjukkan kontainer mana yang sedang aktif menangani *request* dari pengguna pada detik tersebut. Berikut adalah visualisasi tampilan antarmuka saat diakses secara langsung:

a. Tampilan Halaman Web saat diarahkan ke Server 1:

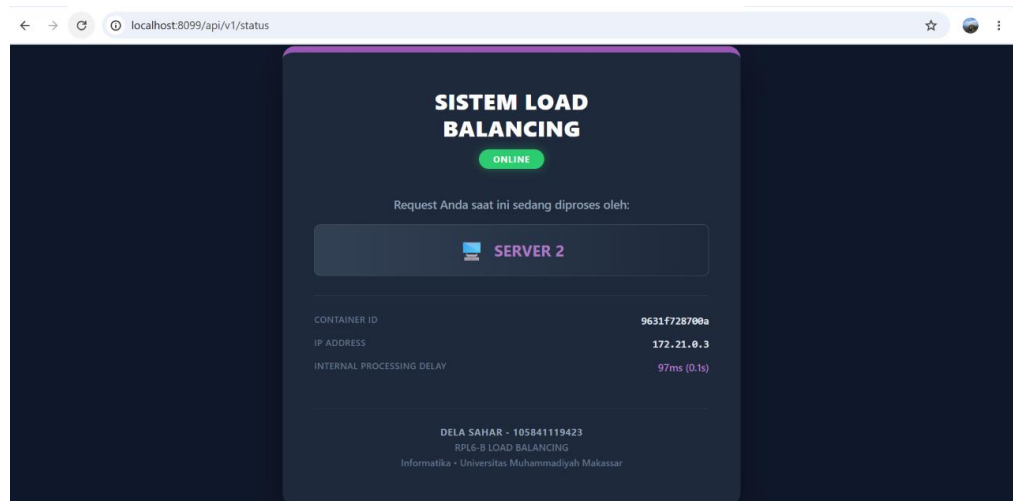
Ketika Nginx mengalihkan paket lalu lintas data ke kontainer pertama, peramban (*browser*) secara dinamis akan merender halaman dasbor premium bertema *Dark Mode* dengan indikator garis aksent berwarna biru cerah yang menandakan status aktif dari Server 1 (**web-node-a**). Di dalam tampilan tersebut, sistem secara otomatis menyajikan informasi spesifik berupa alamat IP internal kontainer, ID unik kontainer sepanjang 12 karakter, serta menampilkan identitas resmi Anda yaitu **DELA SAHAR - 105841119423** sebagai penanda otentik hak kepemilikan sistem yang sedang berjalan.



Gambar 4.3: Tampilan Antarmuka Respon dari Kontainer web 1 pada Browser

b. Tampilan Halaman Web saat diarahkan ke Server 2:

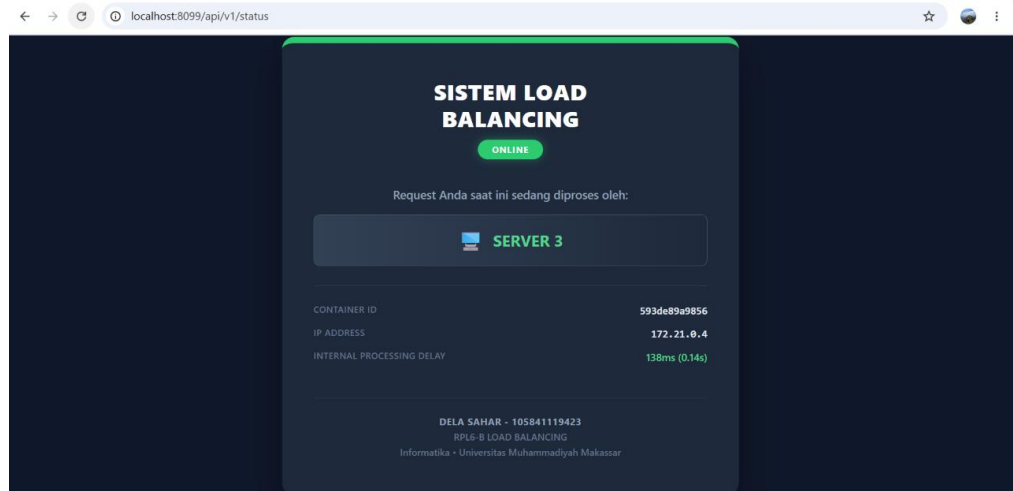
Ketika Nginx mengalihkan paket lalu lintas data ke kontainer kedua saat peramban disegarkan (*refresh*), halaman antarmuka akan otomatis berubah memuat identitas kontainer **web-node-b**. Dasbor premium bertema *Dark Mode* tersebut secara dinamis beralih menampilkan penanda visual bertuliskan **SERVER 2** dengan aksesoris garis indikator berwarna ungu elegan. Pada modul informasi di bawahnya, sistem menyajikan data operasional kontainer secara *real-time*, yang meliputi *Container ID* bernilai 9631f728700a, alamat *IP Address* internal 172.21.0.3, serta informasi *Internal Processing Delay* sebesar 97ms. Seluruh komponen data teknis tersebut disajikan bersamaan dengan identitas resmi Anda, **DELA SAHAR - 105841119423** dari kelas RPL4-B Load Balancing Informatika Universitas Muhammadiyah Makassar, seperti yang diilustrasikan.



Gambar 4.4: Tampilan Antarmuka Respon dari Kontainer web 2 pada Browser

c. Tampilan Halaman Web saat diarahkan ke Server 3:

Selanjutnya, saat peramban kembali disegarkan dan Nginx mengalihkan lalu lintas data ke kontainer ketiga, halaman akan otomatis berubah memuat identitas kontainer **web-node-c**. Dasbor *Dark Mode* tersebut beralih menampilkan penanda visual **SERVER 3** dengan aksen warna hijau *emerald*, lengkap dengan informasi *Container ID*, *IP Address* internal node ketiga (seperti 172.21.0.4), serta nilai *Internal Processing Delay* yang dinamis. Tampilan ini tetap secara konsisten memuat identitas resmi Anda, **DELA SAHAR - 105841119423**, sebagai bukti bahwa kluster ketiga telah sukses melayani permintaan secara seimbang.



Gambar 4.5: Tampilan Antarmuka Respon dari Kontainer web 3 pada Browser

B. Analisis dan Pembahasan

1. Analisis Karakteristik Algoritma Round Robin

Berdasarkan hasil pengujian pada skenario pertama, algoritma *Round Robin* menunjukkan karakteristik utama berupa distribusi beban yang bersifat **deterministik, sirkular, dan merata secara kuantitas**. Nginx membagi setiap permintaan (*request*) yang masuk melalui port 8099 ke seluruh kontainer *backend* (web-node-a, web-node-b, dan web-node-c) secara bergiliran tanpa memedulikan kondisi internal server.

Karakteristik ini memiliki kelebihan dan kekurangan tersendiri dalam penerapannya:

- **Kelebihan:** Sangat adil dalam membagi jumlah antrean permintaan secara numerik dan tidak membutuhkan konsumsi memori tambahan pada *load balancer* untuk memantau status server. Algoritma ini bekerja sangat optimal jika spesifikasi *hardware* tiap kontainer seragam.
- **Kekurangan:** Tidak responsif terhadap fluktuasi latensi (*processing delay*). Ketika salah satu kontainer mengalami lonjakan beban pemrosesan (misalnya latensi acak mendekati 150 ms), Nginx *Round Robin* akan tetap mengirimkan paket data ke kontainer tersebut sesuai gilirannya, sehingga berpotensi memicu penumpukan antrean pada *node* yang sedang sibuk.

2. Analisis Karakteristik Algoritma Least Connections

Berdasarkan hasil pengujian pada skenario kedua dengan perintah `least_conn`, algoritma ini menunjukkan karakteristik utama berupa distribusi beban yang bersifat **dinamis, adaptif, dan berbasis performa riil (*real-time*)**. Nginx tidak lagi membagi beban berdasarkan urutan giliran, melainkan memantau jumlah koneksi aktif pada setiap node kontainer sebelum mengalihkan lalu lintas data.

Karakteristik operasional dari algoritma ini meliputi:

- **Kelebihan:** Sangat efektif dalam menangani fluktuasi beban dan latensi yang tidak merata. Kontainer yang sedang cepat menyelesaikan proses internalnya (latensi rendah ≈ 50 ms) akan otomatis mendominasi penerimaan *request* baru karena status antreannya cepat kosong, sehingga mencegah terjadinya penumpukan data (*congestion*) pada node yang lambat.
- **Kekurangan:** Memerlukan sedikit konsumsi memori dan daya pemrosesan ekstra pada Nginx *Load Balancer* karena sistem harus terus-menerus melacak, menghitung, dan memperbarui status jumlah koneksi aktif dari web-node-a, web-node-b, dan web-node-c secara simultan.

BAB V

KESIMPULAN

Berdasarkan hasil implementasi, pengujian, serta analisis perbandingan sistem *load balancing* menggunakan Nginx dan Docker Container yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Algoritma **Round Robin** terbukti mendistribusikan beban lalu lintas data secara seimbang dan berurutan ke setiap node kontainer (web-node-a, web-node-b, dan web-node-c) secara kaku (sirkular) tanpa mempertimbangkan status latensi maupun kapasitas pemrosesan riil *backend*. Metode ini sangat efisien digunakan pada arsitektur kluster dengan spesifikasi perangkat keras kontainer yang seragam dan menangani beban kerja yang homogen.
2. Algoritma **Least Connections** bekerja secara adaptif dengan memprioritaskan pengalihan lalu lintas data menuju kontainer *backend* yang memiliki jumlah antrean koneksi aktif paling sedikit atau respons tercepat saat itu. Penerapan algoritma ini sangat andal untuk mencegah terjadinya penumpukan beban pemrosesan (*bottleneck*) ketika sistem menerima lonjakan permintaan data (*traffic*) dengan fluktuasi latensi yang tinggi.